THE OPEN SOURCE AUDIT

A Capstone Project — Open Source Software (OSS NGMC)

Chosen Software : **Git**

Student Name : Sarthak Verma

Registration Number : 24BAI10048

Date : 27th March 2026

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Git was born out of absolute necessity in 2005 to solve a massive version control crisis for one of the largest open-source projects in the world: the Linux kernel. Created by Linus Torvalds (the creator of Linux), Git was specifically designed to solve the bottlenecks of older version control systems and handle massive, highly distributed projects.

Before 2005, the Linux kernel development team used a proprietary, closed-source Version Control System (VCS) called BitKeeper. The company behind BitKeeper provided it to the Linux community free of charge. However, after a licensing dispute (a developer tried to reverse-engineer BitKeeper's networking protocols), the free license was abruptly revoked. Linus Torvalds needed a replacement immediately, but he despised the existing free alternatives like CVS and Subversion (SVN).

The Flaws of Centralized Version Control

Older systems like SVN were **Centralized Version Control Systems (CVCS)**. They relied on a single central server that stored all the files and history.

Git was created to solve the massive problems this caused:

- **The Single Point of Failure:** If the central server went down, nobody could commit changes, review history, or collaborate. If the server's hard drive corrupted without a backup, the entire project history was lost.
- **Inability to Work Offline:** Because the history lived on the server, developers could not commit changes or view project history while disconnected from the internet.
- **Slow Performance:** Every time a developer wanted to look at a past commit or compare code (diff), their computer had to talk to the remote server, making things incredibly slow.

The "Branching and Merging" Nightmare

In SVN and CVS, branching was "heavy." Creating a branch meant physically copying all the files into a new directory on the server. Because of this, developers rarely branched, and when they did, merging those branches back together was notoriously difficult and prone to painful conflicts.

The Linux kernel has thousands of developers working simultaneously on different features. Torvalds needed a system that supported non-linear development—where thousands of parallel branches could be created instantly, worked on independently, and merged seamlessly.

Data Corruption and Trust

In older systems, it was possible for files to become corrupted over time or altered without the system noticing. Torvalds wanted absolute cryptographic assurance that the code history hadn't been tampered with.

A2. The license — what it actually says

1. What are the four freedoms of free software? Does your software's license grant all four?

The Four Freedoms of Free Software

The Free Software Foundation defines "free software" by whether it grants users four essential freedoms:

- **Freedom 0:** The freedom to run the program as you wish, for any purpose.
- **Freedom 1:** The freedom to study how the program works and change it so it does your computing as you wish. (Access to the source code is a precondition for this).
- **Freedom 2:** The freedom to redistribute copies so you can help others.
- **Freedom 3:** The freedom to distribute copies of your modified versions to others. By doing this, you can give the whole community a chance to benefit from your changes. (Access to the source code is a precondition for this).

Git is licensed under the GNU General Public License version 2.0 (GPLv2). This specific license was chosen deliberately to protect the open-source nature of the project.

Does Git grant all four? Yes. Because Git uses the GPLv2 license, it explicitly guarantees all four of these freedoms to anyone who downloads it.

2. Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?

No, they cannot. This is the defining feature of the GPL. It is a "copyleft" license. If a company takes Git's source code, modifies it to make a better version, and then distributes or sells that new version to the public, they must release their modified source code under the exact same GPLv2 license.

They are absolutely allowed to charge money for the software, but they cannot legally hide the modified source code from the people they distribute it to. This ensures that the software, and any improvements made to it, remain open and free for the community.

3. What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?

- **GPL (Copyleft):** If someone modifies and distributes your code, they must share their changes under the GPL. It prevents your open-source work from being absorbed into closed-source, proprietary software.
- **MIT (Permissive):** Someone can take your code, modify it, make it closed-source, and sell it without ever sharing their changes with you. They only have to include your original copyright notice and the license text.

Which would I choose?

If I were publishing a Java-based Personal Resource & Study Manager or an HTML/CSS Digital Bookshelf, I would generally choose the **MIT License**.

For those kinds of tools, the goal is usually maximum adoption and usefulness. The MIT license allows other students, developers, or even companies to take the code, learn from it, or integrate it into their own applications without worrying about the strict legal inheritance rules of the GPL. However, if I were building a massive, collaborative infrastructure project (like an operating system or a version control system) where I wanted to guarantee that corporate entities couldn't steal the community's hard work, I would choose the GPL.

4. Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

- **Free as in free beer (Gratis):** The software costs $0. A free mobile game with ads or a proprietary app you download for free is "free beer." You don't pay for it, but you don't own it, and you can't see how it works.
- **Free as in freedom (Libre):** You have the liberty to use, study, modify, and distribute the software.

An example from Git: Git itself was actually born out of this exact distinction. Before 2005, the Linux developers used a version control system called BitKeeper. BitKeeper was "free as in beer"—the company let the Linux developers use it for $0. However, it was *not* "free as in freedom." The code was closed, and developers were legally barred from reverse-engineering it.

When a dispute happened, the company simply revoked the free license, locking the Linux creators out of their own workflow. Linus Torvalds built Git specifically to replace BitKeeper, ensuring that the tool managing Linux would forever be "free as in freedom." Git costs $0, but more importantly, no corporation can ever revoke your right to see its code, change it, or share it.

A3. The ethics of open source

1. Should all software be open source? Make an argument for and against.

The Argument For:

- **Accelerated Innovation:** When code is open, developers worldwide can collaborate, share solutions, and fix bugs instantly. It prevents duplicated effort.
- **Transparency and Security:** There is a famous open-source concept called Linus's Law: "Given enough eyeballs, all bugs are shallow." Closed-source software requires you to blindly trust the creator. Open-source software allows the community to audit the code for privacy violations, malicious trackers, or security vulnerabilities.
- **Democratization:** It lowers the barrier to entry. Anyone with a computer and internet access can study the highest-quality software in the world and learn from it.

The Argument Against:

- **Sustainability and Livelihoods:** Software takes thousands of hours to build and maintain. If everything is free, how do the creators pay their rent? Relying entirely on donations or weekend hobbyists is rarely sustainable for massive, complex systems.

- **Accountability and Liability:** If a piece of open-source software fails and causes a business to lose millions of dollars, there is usually no one to sue and no guaranteed customer support hotline to call.
- **Competitive Advantage:** For companies building highly specialized software (like proprietary trading algorithms or unique video game engines), making the code open source would instantly destroy their business model, as competitors could simply copy it.

2. Is it ethical for a company to build a profitable product entirely on top of free community labor?

It is generally considered ethical because these companies do not just take; they are the biggest financial backers and contributors to the open-source ecosystem.

- **Corporate Contribution:** Companies like Google, Meta, and Microsoft employ thousands of engineers whose full-time, salaried jobs are simply to write and improve open-source code (like Linux, Kubernetes, or React) and give it back to the community.
- **The Red Hat Model:** Red Hat gives its software away for free but charges corporations for enterprise-grade customer support, security patches, and training. They proved that you can build a billion-dollar company without hiding your code.

3. What does 'standing on the shoulders of giants' mean in the context of software? Does open source enable or undermine original innovation?

The phrase "standing on the shoulders of giants" means that our current achievements are only possible because we are building upon the foundational discoveries of those who came before us.

In software, open source is the ultimate embodiment of this idea. When you sit down to write a Java application or deploy a web interface using HTML and CSS, you do not have to write the Java Virtual Machine, invent the compiler, or build the browser rendering engine from scratch. Millions of hours of human intellect have already solved those problems and given the solutions away for free.

Absolutely not, it does not undermine original motivation; moreover, it enables it. Open source handles the heavy lifting of the underlying infrastructure, allowing developers to focus entirely on the unique, original logic of the specific tools they are building—whether that is an interactive digital bookshelf or a customized study manager. If every developer had to reinvent the operating system and the programming language before they could build an application, technological progress would crawl to a halt. We innovate faster exactly because we don't have to start from zero.

Part B — Linux Footprint (Unit 2)

1. Installation Method

Git is universally supported and can be installed via almost any package manager, though compiling from source is also an option for those who need the absolute bleeding-edge features.

- **Debian/Ubuntu (APT):** Installed via sudo apt install git.
- **Red Hat/Fedora (RPM/DNF):** Installed via sudo dnf install git or sudo yum install git.
- **From Source:** You can download the tarball from kernel.org, extract it, and compile it using make prefix=/usr/local all followed by sudo make prefix=/usr/local install.

2. Key Directories

Unlike large enterprise databases, Git is incredibly modular. It spreads its footprint across a few standard Linux directories:

- **Binaries (Executables):** The main executable is almost always located at /usr/bin/git. However, Git relies on dozens of smaller helper programs (like git-commit or git-merge). These core executables are usually tucked away in /usr/libexec/git-core/.
- **Configuration Files:** Git uses a hierarchy of three configuration files:
  - System-wide: /etc/gitconfig (Applies to all users on the Linux machine).
  - User-specific: ~/.gitconfig or ~/.config/git/config (Applies only to your user account).
  - Repository-specific: .git/config (Lives inside the specific project folder you are working on).
- **Logs:** This is where Git differs from typical Linux software. Git does not log to the central /var/log directory. Because Git is highly distributed, all logs (commit history and reflogs) are kept strictly within the project folder itself, specifically under .git/logs/.

3. User and Group Permissions

Git **does not** have a dedicated git system user or group by default (unless you are running a centralized server like GitLab).

Instead, Git runs strictly as the invoking user.

- This is a vital security principle called Least Privilege. Because Git runs as your standard user, if a malicious actor somehow exploited a vulnerability in Git, they would only gain access to your personal files, not the entire root operating system. You should never run Git commands with sudo for everyday project work, as it messes up file ownership and creates security risks.

4. Service Management

Standard Git is a command-line binary, not a daemon or a background service. Therefore, it cannot be managed with systemd commands.

- sudo systemctl start git -> **Will fail.**
- sudo service git status -> **Will fail.**

Git only runs for the few milliseconds it takes to execute a command (like git status) and then exits.

5. The Update Model

When the open-source community fixes a vulnerability or releases a patch, here is how it reaches the machine:

1. **Upstream Patch:** The core Git developers release a patched version (e.g., v2.44.1) on their main repository.
2. **Maintainer Packaging:** Security teams at Canonical (Ubuntu) or Red Hat take that patch, compile it, wrap it into a .deb or .rpm package, and push it to their official distribution repositories.
3. **Local Pull:** Your Linux machine checks those repositories. When you run standard system updates (e.g., sudo apt update && sudo apt upgrade), the package manager pulls the secure version and replaces the old binary in /usr/bin/git.

Part C — The FOSS Ecosystem (Units 3 & 4)

1. What other open-source tools does it depend on?

Git is written primarily in **C** (with some shell and Perl scripts) and relies heavily on a few core, battle-tested open-source libraries to function:

- **zlib:** This handles the compression. When Git stores your history, it compresses the files into tiny, efficient objects using zlib.
- **cURL (libcurl):** This library handles the network protocols (HTTP/HTTPS/FTP). Without cURL, you would not be able to git push or git pull over the web.
- **OpenSSL:** This provides the cryptographic functions necessary for secure connections (like SSH) and generating the SHA-1 hashes that identify your commits.
- **Expat & libiconv:** These handle XML parsing and different text encoding formats so Git can accurately read code written across different operating systems globally.

2. What has it enabled or inspired?

Git completely revolutionized software collaboration and birthed a multi-billion-dollar ecosystem:

- **Hosting Platforms:** GitHub, GitLab, and Bitbucket were built directly around Git's architecture to provide web-based collaboration, issue tracking, and code reviews.
- **CI/CD (Continuous Integration / Continuous Deployment):** Tools like Jenkins, GitHub Actions, and CircleCI rely heavily on Git. As you continue developing your Personal Resource & Study Manager in Java, you could easily set up a CI/CD pipeline that listens to your Git repository and automatically compiles your code and runs your Java unit tests the moment you commit a new feature.
- **GitOps:** This is a modern movement where entire cloud infrastructures (like Kubernetes clusters) are managed using Git. Tools like Argo CD and Flux monitor a Git repository and automatically configure live servers to match the code exactly.

3. Where does it fit in the LAMP stack, or relation to the web?

Git is not formally a component of the classic LAMP stack (Linux, Apache, MySQL, PHP/Python/Perl), but it is the delivery mechanism for it.

In the modern web, developers rarely edit files directly on a live Apache server. Instead, Git sits between the developer and the web server. You write code locally, commit it to a Git repository, and push it. The web server (or a deployment script) then pulls the latest Git commit and serves those files to the internet.

4. Is there a community — mailing lists, forums, conferences, IRC channels? Who governs it?

Unlike many modern projects that use Discord, Slack, or forum software, the core Git community is famously old-school.

- **The Mailing List:** All official Git development, bug reporting, patch submissions, and technical debates happen on a plain-text email mailing list: git@vger.kernel.org.
- **Governance:** Git is not owned by a corporation. After Linus Torvalds created it in 2005, he handed the reins over to Junio C Hamano, who has been the lead maintainer ever since.
- **Contribution Model:** Development is completely decentralized. Contributors email code patches to the mailing list, where they are heavily scrutinized by the community. If the code is solid, Junio or a trusted lieutenant merges it into the core project.
- **Conferences:** The broader community and corporate sponsors (like GitHub and Microsoft) frequently gather at events like Git Merge to discuss the future of the tool and share new ways to optimize it.

Part D — Open Source vs Proprietary (Critical Analysis)

For critical analysis, we're comparing Git against Perforce (Helix Core). Perforce is a heavily utilized, proprietary version control system that dominates enterprise environments dealing with massive binary files (like AAA video game development or semiconductor design). Because its architecture is completely different from Git, it makes for a highly revealing comparison.

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Cost	$0 (Free). Anyone can download, use, and scale it at no cost.	Expensive. Free for very small teams (up to 5 users), but enterprise scaling requires expensive, per-user licensing fees.
Security (who can audit the code?)	Public Audit. Anyone in the world can inspect the source code for vulnerabilities, malicious backdoors, or flaws.	Closed Audit. Only Perforce engineers and specifically vetted, contracted security firms can review the underlying code.
Support and reliability	Community-Driven. Relies on extensive documentation, StackOverflow, and community forums. No guaranteed SLA (Service Level Agreement) or official hotline.	Corporate SLAs. Comes with dedicated, 24/7 technical support, assigned account managers, and guaranteed uptime contracts for enterprises.
Freedom to modify	Absolute. Licensed under GPLv2. You can fork the codebase, rewrite its networking protocols, and customize it completely to your needs.	None. The source code is closed. You are legally restricted to using the API and plugins provided by the vendor.
Community vs corporate control	Community Governance. Steered by a decentralized group of core maintainers and the broader open-source community via public mailing lists.	Corporate Control. The roadmap, feature prioritization, and sunsetting of older versions are entirely dictated by the profit motives of Perforce Software Inc.
Overall verdict	The absolute best choice for standard, text-based software development. It enables rapid, independent branching and fast workflows.	The best choice for monolithic projects requiring strict "file-locking" (preventing two people from editing at once) and massive binary asset management.

Given everything explored, Git is unequivocally recommended for almost all standard real-world deployments. Whether tracking the complex logic of a Java-based resource manager or versioning the layout of a lightweight HTML and CSS web project, Git's distributed nature provides unparalleled speed, safety, and branching flexibility. Its dominance is so absolute that it is a mandatory skill in modern software engineering.

A proprietary alternative like Perforce only becomes necessary in highly specialized, niche environments that require rigid access controls and the management of terabytes of uncompressible graphical assets that Git natively struggles to handle. For a developer, contributing to the FOSS ecosystem is highly recommended. Submitting code to an established open-source tool is one of the most rigorous and rewarding challenges available. It forces a

developer to write optimized code, adhere to strict architectural standards, and navigate professional peer review, ultimately allowing them to give back to the tools that make their own work possible.

Shell Script Tasks

Script 1 : System Identity Report

What this script does : This script serves as a dynamic welcome screen for a Linux environment. It queries the system to retrieve core identity metrics, including the distribution name, kernel version, current user, home directory, system uptime, and the current date and time. It formats these outputs into a clean, readable header and appends a brief statement about the operating system's open-source license.

Concepts used : It utilizes user-defined variables to store static information and command substitution ($()) to execute commands like uname -r, uptime -p, and date, capturing their outputs into variables. It also uses basic output formatting with standard echo commands and string interpolation to construct the final display.

```
sarthakv@INSPIRON-15:~$ nano System_Identity_Report.sh
sarthakv@INSPIRON-15:~$ chmod +x System_Identity_Report.sh
sarthakv@INSPIRON-15:~$ ./System_Identity_Report.sh
====================================================
    OPEN SOURCE AUDIT — System Identity Report
====================================================

  Student  : Sarthak Verma
  Reg No   : 24BAI10048
  Software : Git
====================================================
                SYSTEM INFORMATION
====================================================

OS Distro   : Ubuntu 24.04.4 LTS
Kernel      : 6.6.87.2-microsoft-standard-WSL2
User        : sarthakv
Home Dir    : /home/sarthakv
System Time : 2026-03-26 20:36:24
Uptime      : up 12 minutes
----------------------------------------------------
License Note: The Linux kernel is released under
the GNU General Public License version 2 (GPLv2)
sarthakv@INSPIRON-15:~$
```

Script 2: FOSS Package Inspector

What this script does : This script verifies the installation status of a specific software package (Git) on the system. If the package is found, it extracts and displays key metadata (like Version, License, and Summary). It then passes the package name through a control structure to print a predefined philosophical statement about that specific tool's role in the open-source ecosystem.

Concepts used : It uses an if-then-else statement paired with a silent command execution (&>/dev/null) to check the exit status of the package manager. It uses a pipe (|) and grep -E to filter the verbose output of the package information down to specific keywords. Finally, it uses

a case statement to pattern-match the package name and execute the corresponding echo block. (Note: The starter uses rpm, which is for RedHat/Fedora/CentOS. If you use Ubuntu, replace rpm -q with dpkg -s).

```
sarthakv@INSPIRON-15:~$ nano FOSS_Package_Inspector.sh
sarthakv@INSPIRON-15:~$ chmod +x FOSS_Package_Inspector.sh
sarthakv@INSPIRON-15:~$ ./FOSS_Package_Inspector.sh
================================================================
        FOSS Package Inspector
================================================================

  Checking package: git

  STATUS: git is INSTALLED (Debian-based system)

  Package Details:
  ----------------------------------------------------------------
  Package : git
  Version : 1:2.43.0-1ubuntu7.3
  License  : See /usr/share/doc/git/copyright
  Summary  : fast, scalable, distributed revision control system
 Git is popular version control system designed to handle very large
 projects with speed and efficiency; it is used for many high profile
 open source projects, most notably the Linux kernel.
 .
 Git falls in the category of distributed source code management tools.
 Every Git working directory is a full-fledged repository with full
 revision tracking capabilities, not dependent on network access or a
 central server.
 .
 This package provides the git main components with minimal dependencies.
 Additional functionality, e.g. a graphical user interface and revision
 tree visualizer, tools for interoperating with other VCS's, or a web
 interface, is provided as separate git* packages.

================================================================
  OPEN SOURCE PHILOSOPHY NOTE
================================================================

  Git: Born from frustration — Linus Torvalds built this
  when a proprietary tool (BitKeeper) revoked its free
  license. Git is now the universal language of collaboration,
  proving that open source can outlast any single vendor.

================================================================
sarthakv@INSPIRON-15:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

What this script does : This script automates the auditing of critical system directories and a specific application's footprint. It iterates through a predefined list of paths, checking if each exists. If it does, it extracts the directory's read/write/execute permissions, its owning user and group, and its total disk space consumption. It includes a dedicated check for the global configuration footprint of Git.

Concepts used : It utilizes an array to store the target directories and a for loop to iterate through them. It employs the -d file test operator inside an if-then block to verify directory existence. It heavily utilizes pipes, awk, and cut to parse the raw outputs of ls -ld (for permissions/owners) and du -sh (for disk usage) into clean, isolatable variables.

```
sarthakv@INSPIRON-15:~$ nano Script_3_Disk_and_Permission_Auditor.sh
sarthakv@INSPIRON-15:~$ chmod +x  Script_3_Disk_and_Permission_Auditor.sh
sarthakv@INSPIRON-15:~$ ./Script_3_Disk_and_Permission_Auditor.sh
======================================================================
                      Disk and Permission Auditor
======================================================================

  Auditing standard system directories using a for loop...

  Directory            Permissions/Owner/Group   Size       Status
-----------------------------------------------------------------
  /etc                 drwxr-xr-x root     root     4.3M       OK
  /var/log             drwxrwxr-x root     syslog   129M       OK
  /home                drwxr-xr-x root     root     68K        OK
  /usr/bin             drwxr-xr-x root     root     95M        OK
  /tmp                 drwxrwxrwt root     root     24K        OK
  /var                 drwxr-xr-x root     root     281M       OK
  /opt                 drwxr-xr-x root     root     4.0K       OK

================================================================
  GIT-SPECIFIC CONFIGURATION FILE AUDIT
  (Chosen software: Git)
================================================================

  System config (/etc/gitconfig) : NOT FOUND
  (Git may be installed but no system-wide config set yet)

  User config (/home/sarthakv/.gitconfig) : NOT FOUND
  (Run 'git config --global user.name' to create it)

================================================================
  WHY PERMISSIONS MATTER IN OPEN SOURCE
================================================================

  Linux permissions are a direct expression of open-source values:
  - /etc (755): Readable by all — transparency in configuration
  - /var/log (755/750): Logs visible to admins — auditability
  - /home (755): Each user owns their space — personal freedom
  - /tmp (1777): Sticky bit — shared but protected

================================================================
sarthakv@INSPIRON-15:~$
```

Script 4: Log File Analyzer

What this script does : This script acts as a rudimentary monitoring tool. It takes a log file path as an argument, verifies the file isn't empty, and then scans it line-by-line for a specific keyword (defaulting to "error"). It calculates the total number of keyword occurrences and then fetches the most recent context by displaying the last five lines where the error appeared.

Concepts used : It handles command-line arguments ($1, $2) with default value fallbacks (${2:-"error"}). It uses a while loop combined with [! -s] to simulate a "do-while" retry mechanism, prompting the user until a valid, non-empty file is provided. It then uses a while read -r LINE loop to safely process the file line-by-line, utilizing a counter variable initialized to 0 and incremented via arithmetic expansion ((COUNT + 1)).

```
sarthakv@INSPIRON-15:~$ nano Script_4_Log_Analyzer.sh
sarthakv@INSPIRON-15:~$ chmod +x Script_4_Log_Analyzer.sh
sarthakv@INSPIRON-15:~$ sudo ./Script_4_Log_Analyzer.sh /var/log/syslog
==============================================================
        Log File Analyzer
==============================================================

  Log File : /var/log/syslog
  Keyword  : 'error' (case-insensitive)

  Scanning file...

  --------------------------------------------------------------
  SUMMARY
  --------------------------------------------------------------
  Keyword 'error' found : 15 time(s) in /var/log/syslog

  Last 5 matching lines:
  --------------------------------------------------------------
  [1] 2026-03-26T18:45:25.123377+00:00 INSPIRON-15 kernel: WSL (237) ERROR: CheckConnection: getaddrinfo()
  [2] 2026-03-26T20:24:44.485273+00:00 INSPIRON-15 systemd[1]: apport-autoreport.path - Process error repo
  [3] 2026-03-26T20:24:44.485387+00:00 INSPIRON-15 systemd[1]: apport-autoreport.timer - Process error rep
  [4] 2026-03-26T20:24:44.486715+00:00 INSPIRON-15 kernel: RAS: Correctable Errors collector initialized.
  [5] 2026-03-26T20:25:17.623486+00:00 INSPIRON-15 kernel: WSL (239) ERROR: CheckConnection: getaddrinfo()

==============================================================
sarthakv@INSPIRON-15:~$
```

Script 5: The Open Source Manifesto Generator

What this script does : This interactive script gathers philosophical input from the user regarding their relationship with technology. It asks three tailored questions, captures the responses, and dynamically writes a personalized paragraph to a new text file stamped with the user's name. It then outputs the contents of the newly generated file to the terminal.

Concepts used : It uses the read -p command to prompt the user and capture standard input into variables. It demonstrates string concatenation by blending predefined text blocks with the user's variables into a cohesive paragraph. It uses the >> output redirection operator to append this composed string into a newly created .txt file. Finally, it demonstrates the concept of an alias via an educational comment block.

```
sarthakv@INSPIRON-15:~$ nano Script_5_Open_Source_Manifesto_Generator.sh
sarthakv@INSPIRON-15:~$ chmod +x Script_5_Open_Source_Manifesto_Generator.sh
sarthakv@INSPIRON-15:~$ ./Script_5_Open_Source_Manifesto_Generator.sh
Answer three questions to generate your manifesto.
-----------------------------------------------
1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Accessibility
3. Name one thing you would build and share freely: A universal, interactive tutoring platform

Manifesto successfully saved to manifesto_sarthakv.txt
-----------------------------------------------
===============================================
        MY OPEN SOURCE MANIFESTO
              26 Mar 2026
===============================================

I believe that software should empower, not restrict.
Every day, I rely on tools like Git to learn, create, and grow.
To me, digital rights and open code ultimately mean Accessibility.
In the spirit of standing on the shoulders of giants, my goal
is to give back to the community by eventually building and
freely sharing A universal, interactive tutoring platform so that others may learn from my work.
sarthakv@INSPIRON-15:~$
```